PSAI_146

INK! SMART CONTRACT UPGRADEABILITY

- **Team Name:** Fractal
- **Payment Address:** 0xB341F86A30390048390d6eF118d0e08bBbb0fDef

PROJECT OVERVIEW :PAGE_FACING_UP:

OVERVIEW

The main goal of this project is to bring smart contract upgradeability to ink.

Smart contract upgradeability is an important mechanism to fix security bugs in contracts. It also enables post-deployment iterative development, by offering the flexibility of adding new features.

This mechanism should be flexible enough that contract developers may decide if they want to be able to upgrade their contracts, and specify under what conditions.

Our team is interested in this because it is very likely that our development efforts start with smart contracts before moving to a custom Substrate runtime. Another reason upgradeability is particularly important to us is that we expect to iterate our logic frequently based on a continuous product discovery process.

PROJECT DETAILS

Upgrade functionality can be implemented in user-land, or as part of the smart contracts pallet. We are starting with the first option: it gives us more flexibility to experiment, and iterate faster towards an adequate soltuion. When we're happy with the result, we'll see what can be migrated to ink as a directive to reduce boilerplate.

We're going to start by implementing the proxy pattern. There are a few concerns that will need to be addressed:

- figure out how to handle the separation between state and code
- design a standard for data migrations on contract upgrade
- design a standard for upgrade authorization
- decide if the contract's public interface can/should be upgradable or static

We want to allow for flexibility when deciding if a smart contract can be upgradable or not, and under what circumstances. Should we allow upgrades to the authorization code?

Upgrading the contract's public interface may not be possible when using the proxy pattern.

Ink's documentation and examples provide us with the delegator pattern. This contract works by calling put_code once for the delegator contract and once for the "delegated-to" contract. Every method that is delegated must be explicitly defined in the delegator.

For milestone 1, we will build a template using this pattern with upgrade and authorization methods. This is a template and not a library/ink directive because we want to focus on the upgrade pattern mechanism and leave any ink quality-of-life improvements, like directives or syntax sugar, for the second milestone.

Something that we will consider for future work is extracting any patterns into ink directives that bring us ergonomy of use. For example, assuming we end up deciding on the proxy method, we could automatically generate the proxied methods.

Parts of this work may be liftable to the smart contracts pallet. The advantages of implementing certain features directly in the pallet is that they'd be more efficient and we could tweak their incentive structure.

ECOSYSTEM FIT

Since smart contract development is more accessible and less involved than runtime development, we expect most folks who build on Polkadot to benefit from this mechanism. Substrate offers a natural development path from smart contracts to runtime, so it's also likely that this will assist folks getting started with smart contracts even if they intend to move to a custom runtime in the future.

We didn't find related ideas in the https://github.com/w3f/General-Grants-Program or https://github.com/w3f/Open-Grants-Program repos. We also asked some folks in the community, none of whom were aware of similar projects.

TEAM :BUSTS_IN_SILHOUETTE:

TEAM MEMBERS

- Hugo Peixoto
- Júlio Santos

CONTACT

- **Contact Name:** Júlio Santos
- **Contact Email:** julio@fractal.id
- https://trustfractal.com/

LEGAL STRUCTURE

- **Registered Address:** Wiener Straße 10, 10999 Berlin, Germany
- **Registered Legal Entity:** Trust Fractal GmbH

TEAM'S EXPERIENCE

- Back in 2017, we've implemented a card game (Baccarat) in Solidity. EVM and Solidity have no primitives to generate random numbers, as there is no trustworthy source of entropy available on the blockchain – so we described and built a method for provably fair random number generation with a non-repudiable commitment scheme. This was implemented using state channels for reduced latency and cost. It was built for a client who now owns the IP, so we can't share anything else here, but can do in private if needed.
- In 2017-2018 we built and operated an ICO launchpad to help teams building on blockchain, such as the Ocean Protocol, to crowdfund their projects with community support. See this Medium post from Innogy, an early investor in Fractal, for more information.
- In early 2018 we built a prototype of on-chain KYC using the ERC 725 and ERC 735 standards. See our medium post on this for more information.
- In late 2018, we built our first on-chain KYC integration with DIDs and verifiable credentials, when working with https://ixo.world/. Once we confirmed that a user controlled a certain address (by asking them to sign a nonce), we would create their DID on their behalf. Once their KYC verification completed, we would ledger the corresponding credential against this DID. See our Medium post on this for more information.
- We're currently working with KILT to deliver a decentralised KYC solution for Polkadex. See KILT's medium post on this for more information.

TEAM CODE REPOS

- https://github.com/trustfractal/ink-upgrade-template

TEAM LINKEDIN PROFILES

- https://www.linkedin.com/in/hugopeixoto
- https://www.linkedin.com/in/jvsantos

DEVELOPMENT ROADMAP :NUT_AND_BOLT:

OVERVIEW

- **Estimated Duration:** 1 month
- **FTE:** 0.5
- **Costs:** DAI 9700

MILESTONE 1 – UPGRADABLE INK SMART CONTRACT INFRASTRUCTURE

- **Estimated Duration:** 1 month
- **FTE:** 0.5
- **Costs:** DAI 9700

Number	Deliverable	Specification
0a.	License	Apache 2.0
0b.	Documentation	Documentation on how to use the smart contract template. This should answer the question "how do I build my contract in a way that lets me upgrade it afterwards?"
0c.	Testing Guide	The code will have unit-test coverage (min. 70%) to ensure functionality and robustness. In the guide we will describe how to run these tests
0d.	Article/Tutorial	We will write an article or tutorial that explains the work done as part of the grant.
1.	An Ink! based upgradeable Smart Contract structure	A collection of Ink! smart contracts that be used as a starting point for people that want to have upgradable smart contracts. This collection will contain upgrade logic (including authorization and data migration calls) and some sample methods that users can adapt when developing their own contracts.

FUTURE PLANS

After milestone 1 is complete, we will analyse the resulting smart contracts and see if there is any pattern that would benefit from being extracted onto an ink directive. For example, if we end up using the proxy method, we might propose an ink! directive that automatically generates the delegation methods.

There's also the chance that some patterns or functionality would be better implemented directly into the smart contracts pallet, so that other features, unrelated to upgrades, can take advantage of them.

ADDITIONAL INFORMATION :HEAVY_PLUS_SIGN:

This proposal is slightly research oriented.